Lesson 2

Optimization, Regression, & Classification

> *"Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise."*
>
> — John W. Tukey, *The Future of Data Analysis* (1962)

2.1 Data Preprocessing & Feature Scaling

In theoretical mathematics, we assume the input matrix X is perfectly populated. In physical process engineering, data is extraordinarily messy. Thermocouples short-circuit, pressure transducers experience transient spikes, and data loggers drop packets. If we feed raw, uncleaned physical data directly into our optimization algorithms, they will fail to converge.

Before we can train a machine learning model, we must preprocess the state space $\mathcal{X}$.

2.1.1 Handling Missing Data (Imputation)

When a sensor fails during a batch run, it records a 'NaN' (Not a Number) value. The naive approach is to simply delete any row containing a missing value. However, in the High-Dimension, Low-Sample-Size (HD-LSS) regime, data is too expensive to discard. If we drop every row with a sensor glitch, we might lose part of our training data.

Instead, we use **Imputation**. For independent batch data, we frequently use *Mean Imputation*, replacing the missing value with the arithmetic mean of that sensor across all other batches. For dynamic, continuous time-series data, we use *Forward-Filling*, propagating the last known valid sensor reading forward under the physical assumption that thermodynamic states change smoothly over time.

2.1.2 Outlier Removal (Winsorization)

Physical sensors are prone to transient electronic spikes. If a temperature sensor normally reads 300 K but spikes to 9999 K for one millisecond, it will mathematically destroy our predictive model. Because our objective is to minimize the Squared Error (Equation 1.1), a single massive outlier will dominate the entire loss gradient, forcing the model to warp its physical boundaries just to accommodate the glitch.

To protect the optimizer, we apply **Winsorization** (clipping). We calculate the 1st and 99th percentiles of the sensor's distribution and cap all extreme readings at those boundaries. This preserves the data point without allowing the anomaly to explode the squared error.

2.1.3 Feature Scaling (Standardization)

In a chemical plant, variables exist on radically different scales. A reactor's pressure might be 10^5 Pa, its temperature 300 K, and a catalyst mole fraction 0.01.

If we attempt to numerically optimize our model on unscaled data, the mathematical error space becomes highly distorted. The algorithm will be overly sensitive to the pressure (because the raw numbers are massive) and nearly blind to the mole fraction (because the numbers are microscopic). The resulting error landscape becomes a highly elongated, elliptical ravine. An iterative solver will bounce violently back and forth across the narrow axis while making agonizingly slow progress along the long axis.

To guarantee rapid convergence, we must normalize the geometry of the error landscape using **Z-Score Standardization**. We independently scale every sensor column j so that it has a mean of $\mu = 0$ and a standard deviation of $\sigma = 1$:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{2.1}$$

The Physical Reality: The Geometry of Optimization

Standardization transforms our physical features into dimensionless units (standard deviations from the mean). Geometrically, this transforms the distorted, elongated error landscape into a perfectly symmetrical, spherical bowl. When the landscape is spherical, the numerical optimization algorithms we will derive later in this lesson can point directly at the global minimum, allowing them to converge safely and rapidly in a fraction of the time.

2.2 From Expected Error to Empirical Optimization

In Lesson 1, we established that the optimal predictive model minimizes the Expected Prediction Error (EPE). However, computing the EPE requires knowledge of the true joint probability distribution $P(X, Y)$, which is never known in physical reality.

Instead, we are forced to work with a finite training dataset containing N independent examples: $\mathcal{D} = \{(x_1, y_1), \ldots, (x_N, y_N)\}$. Here, $x_i \in \mathbb{R}^p$ is a column vector representing p distinct sensor readings for the i-th experiment, and $y_i \in \mathbb{R}$ is the continuous scalar target variable we wish to predict. We must transition from theoretical probability to **Empirical Risk Minimization (ERM)**.

2.3 MLE for Linear Regression

How do we mathematically justify drawing a "line of best fit" through a set of noisy reactor data? The answer lies in Maximum Likelihood Estimation (MLE).

Assume our physical target y_i is generated by a deterministic linear combination of our inputs x_i, corrupted by random sensor noise ϵ_i:

$$y_i = x_i^T \beta + \epsilon_i \tag{2.2}$$

where $\beta \in \mathbb{R}^p$ is the column vector of model parameters (weights) we wish to learn, and $x_i^T \beta$ denotes the dot product resulting in a scalar prediction.

The Physical Reality: The Assumption of Gaussian Noise

In process engineering, noise is rarely arbitrary. According to the Central Limit Theorem, if a measurement error is the sum of many small, independent physical fluctuations (e.g., thermal electronic noise in a thermocouple, minor fluid turbulence), the resulting noise distribution converges to a Gaussian (Normal) distribution. Therefore, it is physically rigorous to assume the noise is normally distributed with a mean of zero and variance of σ^2: $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Given the Gaussian noise assumption, the probability density function (PDF) of observing a specific target y_i given the input vector x_i and the parameter vector β is:

$$P(y_i|x_i;\beta,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^T\beta)^2}{2\sigma^2}\right) \tag{2.3}$$

In physical terms, this simply states that a sensor reading is most likely to perfectly match our model's prediction ($x_i^T\beta$), with the probability of measurement errors dropping off exponentially in a symmetric bell curve. For example, if a thermodynamic model predicts a reactor temperature of 400 K, a thermocouple reading of 401 K is highly probable, whereas a reading of 450 K is exponentially less likely, dictated by the sensor's inherent noise variance (σ^2).

The **Likelihood Function**, denoted as $\mathcal{L}(\beta)$, is the joint probability of observing our entire training dataset $\mathcal{D}$. Because we assume each experimental observation (example) is statistically independent, the joint probability is the product of the individual probabilities:

$$\mathcal{L}(\beta) = \prod_{i=1}^{N} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^T\beta)^2}{2\sigma^2}\right) \tag{2.4}$$

MLE dictates that we must find the optimal parameter vector $\hat{\beta}_{\text{MLE}}$ that maximizes this likelihood. Because the natural logarithm (ln) is a strictly monotonically increasing function, maximizing $\mathcal{L}(\beta)$ is mathematically equivalent to maximizing the Log-Likelihood, $\ell(\beta)$. Taking the logarithm

converts the difficult product operator ($\prod$) into a manageable summation operator ($\sum$):

$$\begin{aligned}\ell(\beta) &= \ln\left(\mathcal{L}(\beta)\right) \\ &= \sum_{i=1}^{N} \ln\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - x_i^T\beta)^2}{2\sigma^2}\right)\right) \\ &= \sum_{i=1}^{N}\left[\ln\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{(y_i - x_i^T\beta)^2}{2\sigma^2}\right] \\ &= -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i - x_i^T\beta)^2 \end{aligned} \tag{2.5}$$

To maximize Equation 2.5 with respect to the vector β, we can drop the first term, $-\frac{N}{2}\ln(2\pi\sigma^2)$, because it is a constant that does not depend on β. We can also factor out the positive scaling constant $\frac{1}{2\sigma^2}$. Notice that the remaining summation has a negative sign. Maximizing a negative quantity is strictly equivalent to minimizing the positive quantity:

$$\hat{\beta}_{\text{MLE}} = \underset{\beta}{\text{argmin}} \sum_{i=1}^{N}(y_i - x_i^T\beta)^2 \tag{2.6}$$

Equation 2.6 is the exact definition of the **Ordinary Least Squares (OLS)** objective function. We have just proven a profound mathematical concept: *Minimizing the squared error of a regression model is mathematically equivalent to maximizing the likelihood of the data under the strict physical assumption of Gaussian noise.*

2.4 Gradient Descent as Forward Euler

To solve Equation 2.6, we define our objective (or loss) function $J(\beta)$. In full matrix notation, where $X \in \mathbb{R}^{N\times p}$ is the design matrix containing all inputs and $Y \in \mathbb{R}^{N\times 1}$ is the vector of all targets:

$$J(\beta) = (Y - X\beta)^T(Y - X\beta) \tag{2.7}$$

Expanding this equation and taking the gradient, ∇_β, with respect to β yields:

$$\nabla_\beta J(\beta) = -2X^T(Y - X\beta) \tag{2.8}$$

Setting this gradient to the zero vector gives the closed-form analytical regression solution: $\hat{\beta} = (X^T X)^{-1} X^T Y$. However, for complex non-linear models (like Deep Neural Networks), no such analytical solution exists. Instead, we must rely on iterative numerical optimization techniques.

From a dynamical systems perspective, we can view optimization as a continuous process of moving down an energy landscape (Gradient Flow), governed by the differential equation:

$$\frac{d\beta}{dt} = -\nabla_\beta J(\beta) \tag{2.9}$$

Here, the parameters flow continuously in the exact opposite direction of the gradient. Because computers cannot simulate continuous time, we must discretize this process. Using the **Forward Euler** numerical integration method, we approximate the continuous derivative using a discrete time step $\Delta t = \eta$:

$$\frac{d\beta}{dt} \approx \frac{\beta^{(k+1)} - \beta^{(k)}}{\eta} \tag{2.10}$$

Substituting Equation 2.10 into Equation 2.9 and solving for the next state, $\beta^{(k+1)}$, gives us the exact iterative update rule for **Gradient Descent**:

Definition: Gradient Descent (Forward Euler)

$$\beta^{(k+1)} = \beta^{(k)} - \eta \nabla_\beta J(\beta^{(k)}) \tag{2.11}$$

where $\eta > 0$ is the learning rate (acting mathematically as the discrete time step Δt), $\nabla_\beta J(\beta^{(k)})$ is the gradient vector evaluated at the current parameters, and k is the current iteration step.

Q&A: Why not use the Hessian Matrix?
Question: Forward Euler is merely a first-order approximation. Why don't we use second-order optimization (Newton's Method), which utilizes the Hessian matrix ($H \in \mathbb{R}^{p \times p}$) of second derivatives to find the exact optimal step size?
Answer: Newton's update rule is $\beta^{(k+1)} = \beta^{(k)} - H^{-1}\nabla_\beta J(\beta^{(k)})$. While mathematically precise, computing and inverting the $p \times p$ Hessian matrix requires a computational complexity of $\mathcal{O}(p^3)$. In modern engineering datasets or deep neural networks where the number of parameters p can reach tens of millions, calculating and storing the Hessian is computationally impossible. Therefore, we rely on the first-order approximation (Equation 2.11) with a carefully tuned step size η.

2.5 Logistic Regression for Classification

Not all engineering problems involve predicting a continuous regression value. Often, we must classify a physical state into discrete, mutually exclusive categories: Will the solvent separate into two phases or remain single-phase? Here, our target is binary: $y_i \in \{0, 1\}$.

Linear regression is mathematically invalid here because the output $x_i^T \beta$ can span from $-\infty$ to ∞, predicting probabilities outside the strict $[0, 1]$ bounds required by the axioms of probability. We must map the linear output through a non-linear activation function. The standard choice is the **Sigmoid (Logistic) Function**:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2.12}$$

This function strictly bounds the output between 0 and 1. We now model the predicted probability of the positive class ($Y = 1$) as $\hat{y}_i$:

$$\hat{y}_i = P(Y = 1 | x_i; \beta) = \sigma(x_i^T \beta) = \frac{1}{1 + e^{-x_i^T \beta}} \tag{2.13}$$

Because the output is binary, the data follows a **Bernoulli distribution**. The Probability Mass Function (PMF) of a Bernoulli variable Y given

prediction $\hat{y}_i$ is defined as:

$$P(Y = y_i) = (\hat{y}_i)^{y_i}(1 - \hat{y}_i)^{1-y_i} \tag{2.14}$$

Notice how this formulation works: if $y_i = 1$, the second term becomes $(1 - \hat{y}_i)^0 = 1$, leaving only $\hat{y}_i$. If $y_i = 0$, the first term becomes $(\hat{y}_i)^0 = 1$, leaving only $(1 - \hat{y}_i)$. To find the optimal parameters β, we apply Maximum Likelihood Estimation. We take the product of this probability over all N independent samples, and then take the natural logarithm to form the Log-Likelihood:

$$\begin{aligned} \ell(\beta) &= \ln\left(\prod_{i=1}^{N}(\hat{y}_i)^{y_i}(1 - \hat{y}_i)^{1-y_i}\right) \\ &= \sum_{i=1}^{N}\Big[y_i \ln(\hat{y}_i) + (1 - y_i)\ln(1 - \hat{y}_i)\Big] \end{aligned} \tag{2.15}$$

To frame this as an optimization problem, we want to *minimize* the error rather than maximize the likelihood. We multiply Equation 2.15 by -1 and average it over the N examples. This yields the defining loss function of classification:

Definition: Binary Cross-Entropy (Log-Loss)

$$J(\beta) = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \ln(\hat{y}_i) + (1 - y_i)\ln(1 - \hat{y}_i)\Big] \tag{2.16}$$

2.6 Metrics

Instead of relying on generic accuracy, we map strict statistical metrics directly to physical constraints and system safety. We divide our evaluation into continuous regression metrics and discrete classification metrics.

2.6.1 Regression Metrics

For continuous predictive models, we evaluate the magnitude of the prediction error and the proportion of variance captured by the model.

- **Mean Squared Error (MSE):** Measures the average squared difference between the estimated values ($\hat{Y}_i$) and the actual true values

(Y_i).

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2 \tag{2.17}$$

In thermodynamics, minimizing MSE tightly bounds the physical energy variance. Because errors are squared, MSE heavily penalizes large deviations, which is critical for identifying severe process anomalies.

- **Root Mean Squared Error (RMSE):** The square root of the MSE.

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(Y_i - \hat{Y}_i)^2} \tag{2.18}$$

RMSE is highly valuable in engineering because it transforms the error back into the original physical units of the target variable.

- **Mean Absolute Error (MAE):** Measures the average absolute difference between predictions and true values.

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|Y_i - \hat{Y}_i| \tag{2.19}$$

Unlike MSE and RMSE, MAE applies a linear penalty to errors. It is less sensitive to extreme outliers, providing a robust measure of average baseline deviation without over-penalizing sensor noise.

- **Coefficient of Determination (R^2):** Measures the proportion of the variance in the dependent variable that is predictable from the independent variables.

$$R^2 = 1 - \frac{\sum(Y_i - \hat{Y}_i)^2}{\sum(Y_i - \bar{Y})^2} \tag{2.20}$$

An R^2 of 1.0 indicates perfect predictive tracking of the physical system's dynamics.

2.6.2 Classification Metrics

For discrete events, such as fault detection or safety alarms, predictions are evaluated using a **Confusion Matrix**. This matrix categorizes predictions

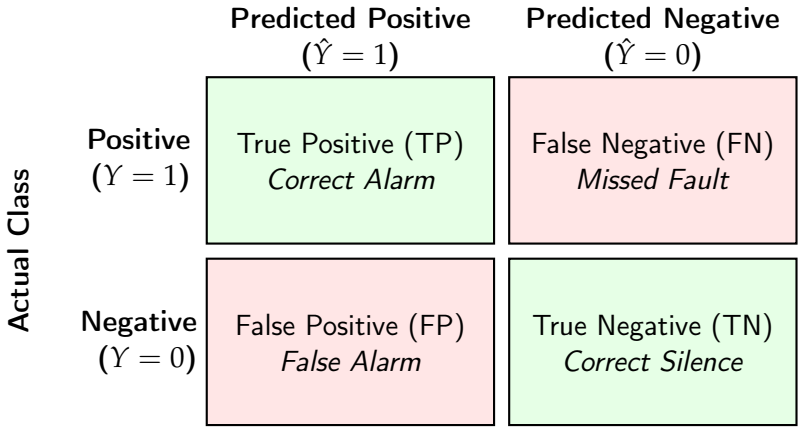


Figure 2.1: Confusion Matrix mapping actual states to model predictions.

into True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN), mapping the model's output to reality.

From this matrix, we derive the following metrics:

- **Precision:** Out of all instances the model flagged as a failure ($\hat{Y} = 1$), the proportion that were actual physical failures. Optimizing precision minimizes False Positives (FP).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{2.21}$$

- **Recall (Sensitivity):** Out of all actual true failures in the physical world ($Y = 1$), the proportion the model successfully detected. Optimizing recall minimizes False Negatives (FN).

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2.22}$$

- F_1**-Score:** The harmonic mean of Precision and Recall. It provides a robust, single-value metric for heavily imbalanced datasets where normal operations vastly outnumber anomaly states.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \tag{2.23}$$

- **Area Under the ROC Curve (AUC)** Machine learning classifiers do not natively output discrete 1s or 0s; they output continuous probabilities, $P(Y = 1|x) \in [0, 1]$. To make a physical control decision (e.g., "trigger the safety alarm"), engineers must define a hard decision threshold, τ. If $P > \tau$, the alarm sounds.
  A low threshold (e.g., $\tau = 0.1$) makes the system hypersensitive, catching every fault (high Recall) but causing massive, expensive False Alarms (high FPR). A high threshold (e.g., $\tau = 0.9$) prevents false alarms but mathematically risks missing a catastrophic failure. The Receiver Operating Characteristic (ROC) curve evaluates the model's structural robustness independent of this specific threshold. It plots the True Positive Rate (Recall) against the False Positive Rate ($\text{FPR} = \frac{\text{FP}}{\text{FP+TN}}$) across *all possible* values of $\tau \in [0, 1]$. The AUC is the mathematical integral (the shaded area) of this curve:

$$\text{AUC} = \int_0^1 \text{Recall}(\text{FPR})\, d(\text{FPR}) \tag{2.24}$$

  An AUC of 1.0 represents a perfect anomaly detector, while 0.5 represents random guessing.

2.6.3 Engineering Trade-offs

Consider an autonomous safety system designed to detect hazardous overpressures in a chemical reactor:

- **Correct Predictions (TP / TN):** The system correctly opens the safety valve during an actual fault, or correctly remains silent during stable operations.
- **False Positive (FP - Low Precision):** *False Alarm.* The valve opens unnecessarily, resulting in halted production, wasted materials, and financial loss.
- **False Negative (FN - Low Recall):** *Missed Fault.* The system fails to detect an active overpressure, resulting in a catastrophic vessel rupture.

In industrial safety, a False Negative is potentially lethal, whereas a False Positive is purely a financial penalty. Therefore, engineering classification decision boundaries must be calibrated to heavily prioritize **Recall**,

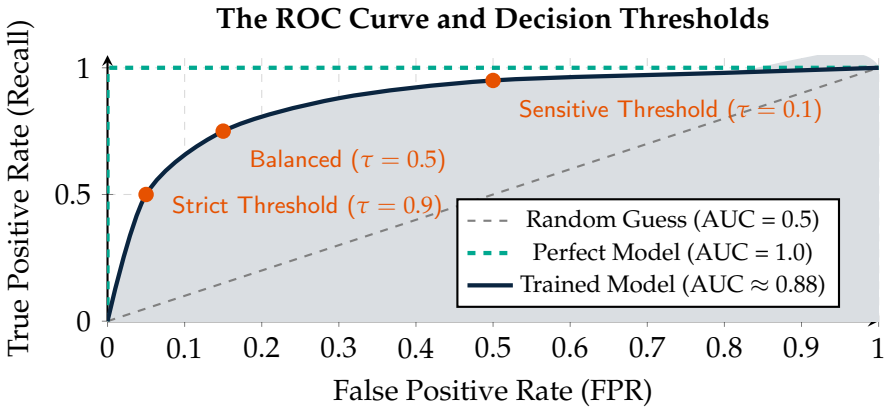


Figure 2.2: ROC curve and threshold trade-offs for safety-critical classification. The shaded area represents AUC, while the marked operating thresholds illustrate how stricter or more sensitive alarm settings change the balance between false positives and false negatives.

ensuring nearly all physical failures are caught, even at the cost of reduced Precision.

2.7 Dataset Splitting & Cross-Validation

Before any optimization begins, the dataset must be divided to ensure the model's generalization ability. A standard data-rich workflow splits the data into three mutually exclusive sets:

- **Training Set (70%):** Used to fit the model and compute gradients.
- **Validation Set (15%):** Held out during training to tune hyperparameters (e.g., learning rate η) and detect the onset of overfitting.
- **Test Set (15%):** A locked vault of data used strictly for the final, unbiased evaluation of the model before physical deployment.

The Physical Reality: The Danger of Data Leakage

A critical, non-negotiable rule of predictive modeling is that **Standardization parameters (μ_j and σ_j) must be computed strictly on the Training Set**.
Once calculated, these exact same training parameters are frozen and applied to scale the Validation Set, the Test Set, and any future real-time plant data. If you calculate the mean and standard deviation using the full dataset before splitting it, information from the "future" (the Test set) mathematically leaks into your training environment. This phenomenon, known as **Data Leakage**, invalidates your safety metrics and guarantees the model will perform drastically worse in physical production than it did on your computer.

2.7.1 k-Fold Cross-Validation

In process engineering, we frequently operate under severe data scarcity. If an engineer only has $N = 40$ destructive materials tests, a 15% test set contains exactly 6 samples. Evaluating a model's safety metrics (like Recall) on 6 samples is statistically meaningless; a single outlier will skew the entire evaluation. To resolve this, we utilize **k-Fold Cross-Validation**. Instead of a single static split, the dataset $\mathcal{D}$ is randomly partitioned into k equal-sized, mutually exclusive folds (commonly $k = 5$ or $k = 10$).

The model is trained k separate times. In each iteration, $k - 1$ folds are combined to form the training set, and the remaining 1 fold acts as the validation set. This rotation continues until every single data point has been used for validation exactly once. The true physical performance of the model is reported as the arithmetic mean of the metric (e.g., MSE or F_1-score) across all k folds:

$$\text{CV}_{(k)} = \frac{1}{k} \sum_{i=1}^{k} \text{Metric}_i \tag{2.25}$$

The Physical Reality: Leave-One-Out Cross-Validation

In extreme High-Dimension, Low-Sample-Size (HD-LSS) environments where $N \leq 20$ (e.g., pilot-plant batch runs), engineers use LOOCV. This is the mathematical limit of k-fold cross-validation where $k = N$. While computationally expensive for large datasets, for small chemical datasets, LOOCV provides an almost unbiased estimate of the true generalization error.

Q&A: Which of the k models do we actually deploy?

Question: After running 5-Fold Cross-Validation, we have 5 different trained models. Which one do we install in the physical plant's control system?

Answer: None of them. Cross-validation is strictly an *evaluation* technique used to reliably estimate the generalization error or to tune hyperparameters. Once the cross-validation process proves the model architecture is safe and accurate, we discard all k intermediate models. We then train one single, final model using the **entire dataset** (100% of the available physical data). This final model, which maximizes the learning from our scarce data, is the one deployed to production.

2.8 The End-to-End Machine Learning Pipeline

A machine learning engineer does not execute these steps randomly. To guarantee physical safety, prevent data leakage, and ensure mathematical convergence, these components must be assembled into a strict, unidirectional **Training Pipeline**.

1. **Data Splitting (First):** Before looking at the data, quarantine a Test Set (e.g., 15%) and lock it away. Divide the remaining data into k folds for cross-validation.
2. **Preprocessing (Fitted on Train Only):** Within each CV loop, fit your Imputation and Standardization parameters (μ, σ) strictly on the training folds. Apply those frozen parameters to scale the validation fold.
3. **Model Initialization:** Define the architecture (e.g., Logistic Regres-

sion) and initialize the weight vector β.

4. **Optimization (Empirical Risk Minimization):** Feed the scaled training data into the optimizer. Compute the loss (e.g., Log-Loss) and update β iteratively using Gradient Descent until the loss converges.
5. **Validation & Tuning:** Evaluate the trained model on the unseen validation fold using strict physical metrics (e.g., Recall, F_1-Score). Adjust hyperparameters (like the learning rate η) if the model is underfitting or overfitting.
6. **Final Testing:** Once cross-validation is complete and the architecture is finalized, train one last model on the entire Training + Validation dataset. Evaluate it exactly once on the locked Test Set. If it passes, the model is ready for physical deployment.

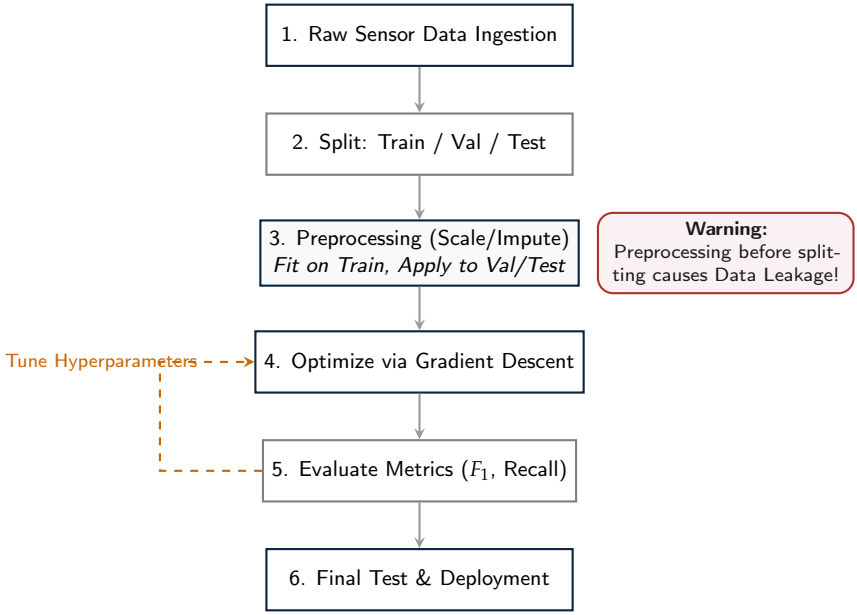


Figure 2.3: End-to-end machine learning training pipeline. The flow emphasizes that splitting the data must happen before preprocessing, and the dashed loop shows how validation results tune hyperparameters without leaking information from validation or test data into training.

2.9 Case Study: Solvent Screening for DAC

Case Study: Phase Behavior and Heat of Absorption

The Setup: A chemical engineer is designing a novel amine solvent for Direct Air Capture (DAC) of CO_2 from the atmosphere.
Cross-Disciplinary Reading: The chemical details are only the setting. Mathematically, this is a two-task engineering problem: first classify whether the system is in a safe state or an undesirable state, and then predict a continuous energy cost. The same structure appears in electronics cooling, materials processing, battery operation, and mechanical equipment monitoring.
The analysis requires two distinct tasks: classifying the phase stability and predicting the energy requirements.
Task 1: VLE Phase Boundary (Classification): Depending on the reactor Temperature (T) and Pressure (P), the solvent either remains a stable liquid (Class 0) or flashes into a two-phase Vapor-Liquid Equilibrium mixture (Class 1), which causes cavitation and destroys the pumps.
Task 2: Heat of Absorption (Regression): For the stable liquid phase, the engineer must predict the heat of absorption (ΔH_{abs}) to accurately size the plant's heat exchangers. Using Linear Regression, they map the continuous relationship between T, P, CO_2 loading, and ΔH_{abs}. Evaluating this model via MSE directly translates to the uncertainty in the required heating duty of the stripper column.

2.10 Summary

In this lesson, we bridged the gap between theoretical probability, raw physical data, and computable optimization:

- **Data Preprocessing:** We established that raw engineering data must be rigorously cleaned before optimization, utilizing Imputation for missing values, Winsorization for transient outliers, and Standardization to ensure a mathematically symmetric loss landscape.
- **Regression vs. Classification:** We defined the operational domains of predictive modeling: utilizing Regression to predict continuous

physical properties (e.g., heat of absorption, enthalpy) and Classification to categorize discrete, mutually exclusive system states (e.g., phase boundaries, safety faults).

- **Mathematical Foundations (Regression):** We proved that assuming physical Gaussian noise and maximizing the likelihood of continuous data (MLE) directly yields the Ordinary Least Squares (OLS) objective (Equation 2.6).
- **Classification Constraints:** To predict discrete probabilities within strict $[0, 1]$ bounds, we utilized the Sigmoid function (Equation 2.12) and derived the strictly convex Binary Cross-Entropy loss (Equation 2.16) directly from the Bernoulli distribution's Probability Mass Function.
- **Numerical Optimization:** We demonstrated that Gradient Descent (Equation 2.11) is equivalent to the Forward Euler numerical integration method, justifying its use over the computationally prohibitive $\mathcal{O}(p^3)$ Hessian matrix.
- **Engineering Evaluation & Validation:** We replaced generic accuracy with strict physical and safety metrics. For continuous regression, we defined MSE, RMSE, MAE, and R^2 to bound energy variance and physical errors. For discrete classification, we utilized the Confusion Matrix to derive Precision, Recall, the F_1-score, and AUC, prioritizing the minimization of catastrophic False Negatives. Finally, we introduced k-Fold and Leave-One-Out Cross-Validation to guarantee robust model generalization even in severely data-limited process environments.